Requerimiento: páginas frontend ASINKO

Sep 23, 2026 · @eugenio

Alcance

Cuatro páginas:

1. **Home** — el feed principal que ve un usuario logueado al entrar a la app.
2. **Comunidades** — una plantilla de página (`/hub/{slug}`) que se adapta según el tipo
    a. grupo
    b. activo financiero
    c. perfil de usuario
3. **Creación de posts** — formulario para publicar un post
4. **Detalle de posts** — la página para verlo con posibilidad de agregar comentarios y votos

Integración con el backend

Todo el consumo de datos va contra la API REST del ASINKO Endpoint Spec (`/api/v1`).

Estado real de la API (repo rama `dev`, commit `2c5cacc`, 23/09/2026) — casi todo lo que estas páginas necesitan ya está implementado, con tests de integración contra base real:

Dominio	Endpoints	Implementado
E01 Identidad y cuenta	11	11/11 ✅
E02 Comunidades	10	10/10 ✅
E03 Posts y encuestas	7	7/7 ✅
E05 Interacción social (votos, comentarios)	15	15/15 ✅
E04 Feed y descubrimiento	3	3/3 ✅
E07 Notificaciones	5	0/5
E11 Thesis	6	5/6 — falta solo el voto de resolución comunitaria, fuera de MVP por diseño
E12 Track record	1	1/1 ✅
E14 Primitivas financieras	8	0/8

Impacto en las 4 páginas de este documento:

- **Home**, **crear post** y **detalle de post**: el backend está completo — ninguno tiene bloqueo.

- **Comunidades**: completo, incluidos favoritos (`PUT`/`DELETE /communities/{slug}/favorite`) y `GET /communities?mine=true` para la lista de "mis comunidades".
- **Limitación conocida**: la página de activo se construye sin datos de mercado. Precio, sentiment y alertas dependen de E14, postergada hasta que haya proveedor de precios.

Auth: sesión de Supabase vía cookie. Toda escritura exige un `userId` explícito que el backend compara contra la sesión real.

Listas: paginadas por cursor, `{ items, nextCursor }` — nunca por página/offset.

Cómo arrancar el entorno

El paso a paso completo está en `apps/web/README.md`. Lo esencial:

1. `npm ci` desde la raíz del repo — es un monorepo con npm workspaces, no se instala por paquete.
2. Base de datos: Postgres local, `packages/db/.env` con `DATABASE_URL` y `DIRECT_URL`, y después `npm run migrate:dev --workspace=packages/db`.
3. Datos de prueba: `npm run seed --workspace=packages/db` — deja 10 perfiles con su canal personal, 12 comunidades, posts, encuestas, tesis, follows y favoritos.
4. Auth: `apps/web/.env.local` con `NEXT_PUBLIC_SUPABASE_URL` y `NEXT_PUBLIC_SUPABASE_ANON_KEY` (formato en `apps/web/.env.example`; los valores los pasás vos por canal seguro). **Sin esas dos variables falla toda request**, incluidas las páginas públicas: el proxy crea el cliente de Supabase en cada una.
5. `npm run dev --workspace=apps/web`.

Tres cosas que conviene saber antes de empezar:

- Los usuarios del seed (`maria`, `jeppe`, `martin`…) **no pueden iniciar sesión**: no son cuentas de Supabase Auth, sirven para ver contenido. Para trabajar como usuario real hay que registrarse — el login con Google ya funciona de punta a punta.
- **Publicar exige ser miembro de la comunidad** (`403 NOT_COMMUNITY_MEMBER` si no lo sos). Un usuario recién registrado solo tiene su canal personal: primero unirse a alguna comunidad con `PUT /communities/{slug}/members/{userId}`.
- `API-IMPLEMENTATION-NOTES.md`, en la raíz del repo, lista dónde la implementación se aparta del Endpoint Spec. Conviene leerlo antes de programar contra el spec.

Home — feed del usuario

Propósito: lo primero que ve un usuario logueado. **En esta etapa el feed muestra solo posts** — el único tipo de contenido que se pide crear en este documento.

Estado actual: `/` (`apps/web/src/app/page.tsx`) ya renderiza esto en producción con datos reales, pero Prisma directo, no la API. Lo que falta es interactividad, no la estructura: los tabs *Para Ti*/*Siguiendo* y *Hot*/*New*/*Top* son texto estático sin wiring, "+ Crear" y las flechas de voto son `<span>` inertes, y la campana de notificaciones no hace nada. Hay mockup Hi-Fi en `UX/mockups/` (flujo "Home feed") y los componentes `PostCard`/`PollCard`/`ThesisCard` ya muestran los campos reales — partir de ahí.

Estructura de contenido:

- Dos vistas: *Siguiendo* (`GET /feed/following`, requiere sesión) y *Explorar* (`GET /feed/explore`, pública) — tabs o toggle entre ambas.
- Cada item es un `FeedItem` (`{ kind: POST|THESIS|POLL, content }` — ver la entidad completa en el Endpoint Spec, sección Entities). **En esta etapa solo se dibuja la tarjeta de `kind: "POST"`**: título, cuerpo (o preview si es largo), imagen si `type=IMAGE`, autor (o "ASINKO" si `creatorType=SYSTEM`), score, cantidad de comentarios, voto del viewer.
- **Tesis y encuestas quedan para más adelante.** Existen en el modelo y la API ya las devuelve (`kind: "THESIS"` y `"POLL"`), con su propia tarjeta e interacciones — **no hay que construirlas ahora**. Vale la pena que el switch por `kind` deje el lugar hecho, para que sumarlas después no obligue a rehacer el feed.
- **A resolver con backend antes de empezar:** hoy los dos endpoints devuelven los tres tipos mezclados y no hay filtro por tipo. Descartar tesis y encuestas en el cliente rompe el tamaño de página (una página de 20 puede quedar en 12 y la paginación se vuelve impredecible). Lo razonable es pedir un `?kind=POST` en la API.
- Paginación por cursor (`nextCursor`): scroll infinito o botón "cargar más".
- Estado vacío: usuario sin comunidades seguidas → invitar a explorar o seguir comunidades (el CTA nada más, no hace falta diseñar el flujo de seguir acá).
- Estado de carga: skeleton acorde al mockup, no un spinner genérico.

Interacciones que debe soportar:

- Votar / quitar voto en un post (`PUT/DELETE /posts/{id}/vote`).
- Votar tesis y encuestas queda para cuando entren esos tipos; los endpoints ya existen (`PUT/DELETE /theses/{id}/vote`, `PUT /polls/{id}/vote`).
- Votar requiere sesión — sin sesión, la UI debe indicarlo (por ejemplo, redirigir a login) en vez de fallar silenciosamente.

Endpoints que consume esta página: `GET /feed/following` · `GET /feed/explore` · `PUT/DELETE /posts/{id}/vote`

Páginas de comunidad, activo y perfil

Propósito: tres páginas con estructura parecida — todas son una `Community` por debajo — pero **cada una con su URL propia**, según el inventario de pantallas de la IA (`UX/information-architecture.md` §3). Compartir componentes internamente está bien; compartir la ruta no.

Página	URL	`Community.type`	Qué es
Comunidad	`/hub/{slug}`	`USER_PUBLIC`	Comunidad temática creada por un usuario
Activo	`/asset/{symbol}`	`SYSTEM_ASSET`	Ticker page de un activo financiero

Página	URL	`Community.type`	Qué es
Perfil	`/user/{username}`	`PERSONAL_CHANNEL`	El espacio público de un usuario (Canal Personal)

Las tres rutas están fijadas fuera de este documento y conviene respetarlas: el design-system linkea los chips inline `$AAPL` a `/asset/AAPL` y `@federico` a `/user/federico`, y E13-01-03 define la URL pública del Canal Personal como `/user/{username}`. Si las tres vivieran bajo `/hub/`, esos links apuntarían a rutas inexistentes.

Referencia visual: el mockup llega por separado — este documento cubre qué datos y funcionalidad debe soportar cada variante, no el layout.

Estado actual: ninguna de las tres rutas existe en el repo — se construyen de cero. El backend de comunidades está **completo** (lectura, edición, membresías, roles, favoritos, `?mine=true`) y el de perfiles también. El de activos no: ver la página de activo más abajo.

Común a las tres (vía `GET /communities/{slug}`, más el recurso propio de cada una):

- Nombre, descripción, avatar/banner, cantidad de miembros.
- Estado del viewer: `viewerMembership` (`null`/`MEMBER`/`MODERATOR`/`PRINCIPAL_MODERATOR`) y `viewerIsFavorite` — botón de unirse/salir (`PUT`/`DELETE` `/communities/{slug}/members/{targetUserId}`) y de favorito (`PUT`/`DELETE` `/communities/{slug}/favorite`).
- Listado de miembros (`GET /communities/{slug}/members`), si el diseño lo pide.
- El feed propio de la comunidad ya tiene endpoint: `GET /feed/explore?community={slug}` devuelve el contenido de una sola comunidad, con el mismo `FeedItem` y la misma paginación por cursor que la Home.

Página de perfil — `/user/{username}`:

- Perfil completo vía `GET /profiles/{identifier}` (acepta `username` o `userId`): bio, `badgeStatus`, `accuracyRate`, `currentStreak`, contadores de correctas/incorrectas/canceladas, seguidores/seguidos.
- "Seguir" a este usuario es un alias documentado de unirse a su `PERSONAL_CHANNEL` — misma acción que el botón de unirse común a las 3 variantes.
- Historial de tesis del usuario: `GET /users/{userId}/theses`.

Página de activo — `/asset/{symbol}`: se construye en esta etapa, **sin datos de mercado**. Todo lo que no dependa de precio ya se puede armar con los endpoints de comunidad.

- **Cómo resolver la ruta:** cada activo tiene su comunidad `SYSTEM_ASSET`, y su `slug` **es el símbolo** (`AAPL`, `BTC`, `NVDA`, `ETH`, `VOO` en el seed). O sea, `/asset/AAPL` → `GET /communities/AAPL`, sin endpoint intermedio. De ahí salen nombre (`$AAPL`), descripción, avatar, `memberCount`, `viewerMembership` y `assetId`.
- **Contenido del activo:** `GET /feed/explore?community={symbol}` — el feed de esa comunidad, con el mismo `FeedItem` y la misma paginación que Home (y, como en Home, por ahora solo se dibujan los posts).

- **Acciones:** unirse/salir y favoritear funcionan igual que en cualquier comunidad.
- **Fuera de esta etapa por falta de proveedor de precios y datos financieros:** precio, variación del día, `priceStale`, el sentiment agregado Bullish/Bearish y las alertas de precio. Esos endpoints (`GET /assets/{symbol}`, `PUT/DELETE /assets/{symbol}/sentiment`, `/price-alerts`) **no existen todavía** — no programar contra ellos.
- Para esa parte de la UI: dejar el espacio previsto con un estado vacío explícito ("sin datos de mercado todavía"), nunca valores inventados ni de ejemplo.

Página de comunidad — `/hub/{slug}`:

- Reglas de la comunidad (`rules`), link externo (`mainUrl`) si existe.
- Si el viewer es `MODERATOR+`, acciones de gestión: editar comunidad (`PATCH /communities/{slug}`), cambiar rol de un miembro (`PATCH /communities/{slug}/members/{targetUserId}`), remover miembro (`DELETE /communities/{slug}/members/{targetUserId}`) — a confirmar si esta primera etapa las incluye o quedan para después.

Endpoints que consumen estas páginas: `GET /communities/{slug}` · `GET /communities/{slug}/members` · `PUT/DELETE /communities/{slug}/members/{targetUserId}` · `PUT/DELETE /communities/{slug}/favorite` · `GET /feed/explore?community={slug}` · `GET /profiles/{identifier}` · `GET /users/{userId}/theses` · `PATCH /communities/{slug}` (si aplica gestión)

Creación y detalle de posts

Estado actual: no hay página de creación ni de detalle en el repo — se construyen de cero. El backend está **completo**, incluidos los comentarios y votos de E05 que antes eran el bloqueante: árbol de comentarios anidado, crear y responder, y votar post, tesis o comentario.

Crear post — formulario:

Campo	Notas
`communityId`	requerido — a qué comunidad se publica
`type`	requerido — `TEXT`, `LINK` o `IMAGE`
`title`	requerido, ≤300 caracteres
`body`	requerido, markdown
`url`	solo si `type = LINK`
`imageUrl`	solo si `type = IMAGE` (subida de imagen: definir con quien mantiene la API dónde vive el upload — el Endpoint Spec solo recibe la URL final)
`tag`	opcional

Ojo con dos cosas de `POST /posts`. La detección de `$TICKER`/`@mention`/`#comunidad` **está postergada**: el campo `entities` viaja siempre vacío, así que por ahora esos links no se pueden renderizar (ni conviene parsearlos en el cliente, para no tener que deshacerlo después). Y publicar **exige ser miembro** de la comunidad: `403 NOT_COMMUNITY_MEMBER` si no lo sos, o `403 PERSONAL_CHANNEL_OWNER_ONLY` si es el canal personal de otro. Errores de validación: `422` si `title`/`body` están vacíos o exceden el límite, o si `communityId` no existe.

Tras crear, redirigir al detalle del post nuevo.

Detalle de post (`GET /posts/{id}`, público si la comunidad es pública):

- Post completo: título, cuerpo, imagen/link según `type`, autor, score, `viewerVote`.
- Votar / quitar voto (`PUT`/`DELETE /posts/{id}/vote`).
- Árbol de comentarios completo, sin paginar (`GET /posts/{id}/comments`): cada comentario trae `replies` anidado.
- Crear comentario o respuesta (`POST /posts/{id}/comments`, con `parentCommentId` opcional para anidar).
- Votar / quitar voto en un comentario (`PUT`/`DELETE /comments/{id}/vote`).
- Editar/borrar post y editar/borrar comentario quedan fuera de esta primera etapa (ver Fuera de alcance).

Endpoints que consume esta página: `POST /posts` · `GET /posts/{id}` · `PUT`/`DELETE /posts/{id}/vote` · `GET /posts/{id}/comments` · `POST /posts/{id}/comments` · `PUT`/`DELETE /comments/{id}/vote`

Fuera de alcance (para esta primera etapa)

- Crear tesis o encuestas (solo posts entran en esta etapa).
- Editar o borrar posts y comentarios.
- Notificaciones.
- Búsqueda (`GET /search`).
- Moderación de contenido más allá de lo listado en la variante Grupo de Comunidades.
- Alta de usuario y login — fuera de alcance desde el 23/09/2026. El flujo ya existe y funciona en el repo (`/login`, `/onboarding/username`), incluido el ingreso con Google.

Checklist de entregables

- ☐ Home renderiza posts con datos reales de la API (no mock estático) en ambas vistas (Siguiendo/Explorar), con paginación por cursor y sus 3 estados (cargando, vacío, con contenido).
- ☐ Votar / quitar voto funciona en un post desde el feed y desde su detalle.
- ☐ Las tres páginas resuelven y renderizan en su ruta propia: `/hub/{slug}` con un grupo `USER_PUBLIC` del seed, `/user/{username}` con un usuario, y `/asset/{symbol}` con un activo (`AAPL`, `BTC`).
- ☐ Unirse/salir y favoritear una comunidad funciona y refleja el estado en la UI sin recargar.
- ☐ Crear un post funciona de punta a punta y redirige a su detalle.
- ☐ El detalle de un post muestra el árbol de comentarios completo, permite comentar/responder, y votar post y comentarios.

- ☐ Todas las páginas funcionan sin sesión (estado público) y con sesión (estado personalizado) donde el Endpoint Spec lo distingue.
- ☐ Responsive: funciona en mobile.